Lab Exercise
Code Quality and Design Quality

Individual homework, due Sunday 25 October 2026 at 23:55

Summary

In this lab, you check the same Java code with two tools and compare what each tool finds. SonarQube for IDE checks code quality, such as probable bugs and code smells. An AI review prompt checks code quality and also design quality in the terms of this course, namely cohesion, coupling and connascence. You then ask an AI to write one small program twice, once with a poor design and once with a good design, and you check whether each tool can tell the two apart.

Your Project Phase 2 report records the results of the same two tools, so this lab also prepares you for the quality checks of Phase 2. Follow the steps in order. Part A installs the tools, which you also need for Phase 2, Part B creates your repository, and Parts C to G are the five tasks.

What You Submit

- A public GitHub repository that holds your work in the layout below. Anyone can read a public repository, so do not write your name or student ID anywhere in it. MyCourses links your submission to you.

```
<your repository>/
    .gitignore          from the starter files
    LAB_REPORT.md       your report, from the starter files
    prompts.md          the prompts, from the starter files
    parcelpoint/        the starter project, with your fixes
    bikeshare-poor/     the poor design of Task 4
    bikeshare-good/     the good design of Task 4
    reviews/            the full output of every AI review run
```

- The completed LAB_REPORT.md at the root of the repository.
- On MyCourses, the URL of your repository and the file LAB_REPORT.md, before Sunday 25 October 2026 at 23:55.
- This is individual work, so copying another student's report or code counts as plagiarism. Report any problem with the tools to the instructor before the deadline.

Level of AI Usage

Level 3: AI Collaboration

Tasks 2 and 4 require an AI tool, and you may also use AI in other parts of the lab. Record the tool, the exact model and the date of every use in the report. Check every AI answer yourself, because an AI can be wrong, and the report asks you to show how you checked. Use the prompts as given, and if you change a prompt, copy the changed prompt in full into the report.

Part A. Install the Tools

Skip any step whose tool is already installed, but still run its check command. In the steps below, a terminal means Command Prompt or PowerShell on Windows, and the Terminal app on macOS. Always open a new terminal after you install something, so that it finds the new tool.

A1. Java Development Kit (JDK), version 17 or later

1. Open https://adoptium.net in a web browser, and click Download Temurin. Download the installer of a version marked LTS, 17 or later, which is the .msi file on Windows or the .pkg file on macOS.
2. Run the installer. On Windows, on the Custom Setup page, click Set JAVA_HOME variable, choose Will be installed on local hard drive, and finish the installer with the default options. On macOS, accept the default options.
3. Open a new terminal, type the command below, and press Enter. The first line of the output must show version 17 or later.

```
java -version
```

A2. Maven

1. On Windows, open https://maven.apache.org/download.cgi. In the first table, which is for the current release, click the link that ends in -bin.zip, such as apache-maven-3.10.0-bin.zip. Unzip it into a folder whose path has no spaces, such as C:\tools, which gives a folder whose name starts with apache-maven.
2. On Windows, click Start, type environment variables, and open Edit the system environment variables. Click Environment Variables. Under User variables, select Path and click Edit. Click New, type the full path of the bin folder inside the Maven folder, such as C:\tools\apache-maven-<version>\bin, and click OK in each of the three windows.
3. On macOS with Homebrew, run the command below in the terminal.

```
brew install maven
```

4. On macOS without Homebrew, download the file that ends in -bin.tar.gz from the same table, and double-click it in Finder to unpack it. Move the unpacked folder, whose name starts with apache-maven, into your home folder, which Finder opens with Go, then Home. Then run the command below in the terminal, with the name of that folder in place of apache-maven-<version>.

```
echo 'export PATH="$HOME/apache-maven-<version>/bin:$PATH"' >> ~/.zshrc
```

5. Open a new terminal and run the command below. The output must show the Maven version, and Java version 17 or later.

```
mvn -version
```

A3. Git and GitHub

1. Check that you can sign in at https://github.com with the account that you used in Project Phase 1.
2. On Windows, install Git from https://git-scm.com/downloads with the default options. On macOS, run the command of step 3 in the terminal. If macOS offers to install the command line developer tools, click Install, and run the command again when the installation ends.
3. Open a new terminal and run the command below. The output must show a version number.

```
git --version
```

4. Run the two commands below one at a time, with your GitHub username and an email address between the quotes. Git records both in every commit, and anyone can read them in a public repository. To keep your email private, use the address ending in users.noreply.github.com that GitHub shows under Settings, then Emails.

```
git config --global user.name "your-github-username"
git config --global user.email "you@example.com"
```

A4. VS Code and its two extensions

1. Install VS Code from https://code.visualstudio.com and open it.
2. Click View, then Extensions. In the search box, type Extension Pack for Java, click the result whose publisher is Microsoft, and click Install.
3. In the same search box, type SonarQube for IDE, click the result whose publisher is SonarSource, and click Install.
4. Click View, then Command Palette, type SonarQube: Enable Automatic Analysis, and press Enter. If no such command appears, automatic analysis is already on. SonarQube for IDE then checks every Java file that you open.

Part B. Create Your Repository

B1. Create the repository on GitHub

1. Sign in at https://github.com, click the + button at the top right, and click New repository.
2. In Repository name, type itcs382-lab. Choose Public, leave every other option unchanged, and click Create repository.
3. On the page that opens, find the Quick setup box, make sure HTTPS is selected, and click the copy button next to the address, which ends in .git.

B2. Open the repository in VS Code

1. In VS Code, click View, then Command Palette, type Git: Clone, and press Enter.
2. Paste the address that you copied, and press Enter. Choose a folder that OneDrive or iCloud does not sync, such as C:\projects on Windows or a folder named projects in your home folder on macOS, because syncing can interfere with the files that Git and Maven write. Create the folder with the New Folder button of the dialog if it does not exist, and click Select as Repository Destination. Sign in to GitHub in the browser if VS Code asks you to.
3. When VS Code asks whether to open the cloned repository, click Open. If VS Code asks whether you trust the authors of the files in this folder, click Yes, I trust the authors.

B3. Add the starter files

1. Download the starter zip from MyCourses and unzip it. Inside it is a folder named ITCS382_Lab_Code_and_Design_Quality.
2. Open that folder in File Explorer on Windows, or in Finder on macOS. On macOS, press Command+Shift+. first, so that Finder also shows the file .gitignore, whose name starts with a dot. Copy the four items inside, namely the parcelpoint folder, prompts.md, LAB_REPORT.md and .gitignore, and paste them into your repository folder. That folder has the name of your repository and sits in the folder that you chose in step B2.
3. In VS Code, check that the Explorer on the left shows the four items. If .gitignore is missing, right-click an empty area of the Explorer, click New File, name it .gitignore, type the two lines below, and save it with Ctrl+S, or Command+S on macOS.

```
target/
*.csv
```

B4. Commit the starter before you change anything

1. Click View, then Source Control. Type Starter code in the Message box, and click Commit. If VS Code asks whether to stage all your changes and commit them directly, click Yes.
2. Click Publish Branch or Sync Changes, whichever button VS Code shows.
3. Click Terminal, then New Terminal, and run the command below. The line that it prints starts with a code of seven letters and digits, such as 3f9c2ab. Open LAB_REPORT.md, write that code on the line Commit of the starter, before any change, under Task 1, and save the file.

```
git log --oneline -1
```

4. Open your repository on GitHub in the browser, and check that the files are there.

Part C. Task 1, SonarQube on the Starter Code

ParcelPoint runs the parcel lockers on a university campus. The starter project stores parcels in lockers, lets students pick them up and pay a fee, and saves the parcels to the file parcels.csv.

C1. Wait for the Java project to load

1. Look at the status bar at the bottom of the VS Code window, and wait until it shows Java: Ready. The first time can take a few minutes.
2. If it shows Java: Lightweight Mode instead, click View, then Command Palette, run Java: Switch to Standard Mode, and wait for Java: Ready.

C2. Run the program

1. In the terminal, run the three commands below one at a time. The terminal opens in your repository folder. The first mvn package downloads what Maven needs, which can take a few minutes, and it must end with BUILD SUCCESS before you run the java command.

```
cd parcelpoint
mvn package
java -cp target/classes parcelpoint.Main
```

2. When the program prints its list of commands, type the four lines below, and press Enter after each one.

```
store P1 6501 2
pickup P1 5
staff
quit
```

3. Look at the output, and click parcels.csv in the parcelpoint folder of the Explorer to see what the program saved. Note anything that looks wrong, and leave the code as it is, because you fix it in Task 3.
4. Run cd .. to bring the terminal back to the repository folder.

C3. Collect the SonarQube issues

1. In the Explorer, click the folder parcelpoint, then the folder src/main/java/parcelpoint, which VS Code may show as one row. Double-click each of the four Java files, so that each stays open in its own tab.
2. Click View, then Problems. Each SonarQube issue shows its message, the rule in brackets, such as sonarqube(java:S1068), and its position, such as [Ln 12, Col 17]. The panel also lists warnings from the Java extension, which show Java in place of sonarqube, such as Resource leak: 'in' is never closed. Leave those out of Table 1.
3. If the Problems panel shows a warning that a file is a non-project file, the Java project did not load. Click File, then Open Folder, choose the parcelpoint folder inside your repository, and click Open. Wait for Java: Ready, and double-click the four Java files again. When Table 1 is complete, open your repository folder again in the same way, because the later parts work from it.
4. To read why a rule matters, click the underlined code in the editor, press Ctrl+., or Command+. on macOS, and choose Show issue details.
5. Copy every SonarQube issue into Table 1 of LAB_REPORT.md, with its rule, file, line and message. Leave none out. To add a row to a table, copy its last row onto the next line and change the number.

Part D. Task 2, AI Review of the Starter Code

1. Open prompts.md, and copy the lines of Prompt A that lie between the line ```text and the next line ```. In the copied text, replace <folder> with parcelpoint.
2. Open your AI tool, such as ChatGPT, Gemini or Copilot Chat, and start a new chat. Click the attach button next to the message box, which is usually a paperclip or a plus sign, and attach the four Java files from

parcelpoint/src/main/java/parcelpoint. Paste Prompt A into the message box and send it. If your tool cannot attach files, paste each file after the prompt instead, with a line such as File: Parcel.java above it.

3. Copy the whole answer with the copy button under it. In VS Code, right-click an empty area of the Explorer, click New Folder, and name it reviews. Right-click reviews, click New File, name it parcelpoint-review.md, paste the answer, and save the file.
4. In the first table of Task 2 in the report, write the date, the AI tool, the exact model name, the commit of the starter and how the AI read the files. Most tools show the model name at the top of the chat or in a model menu.
5. Fill Table 2 with one row for each problem that either tool found. When both tools found the same problem, write one row and mark it both. An example row from another program reads Shop.java:12, Unused field total, SonarQube, java:S1068, yes because nothing reads total.
6. For each AI finding, open the file at the line that the AI gives, and decide whether the finding is correct. Write yes or no in the last column, with one reason.
7. Answer questions 2.1 to 2.3 in the report.

Part E. Task 3, Fix and Check

1. Fix the issues of Table 1 one at a time, and save the file after each fix. When a fix is right, its issue disappears from the Problems panel.
2. Keep an issue only when you can give a one-sentence reason. For example, printing in Main is the output of a command-line program, so you may keep it with that reason.
3. Fix two design problems from Table 2 that you agree with. Examples are a class that does more than one job, and a number code, such as the parcel size, that two classes must read in the same way. Keep each change small.
4. After each fix, run the commands below in the terminal to check that the program still builds and runs. Type quit to end it.

```
cd parcelpoint
mvn package
java -cp target/classes parcelpoint.Main
cd ..
```

5. Commit each fix in the Source Control view with a short message that says what you fixed, such as Fix string comparison in pickUp, and click Sync Changes. Run the git log command of step B4 again, and copy the code into Table 3.
6. Finally, run the program with the four lines of input from step C2, and copy its output and the content of parcels.csv into the report.

Part F. Task 4, AI-Generated Poor and Good Designs

The BikeShare specification in prompts.md describes a small bike lending program for the campus. You create two Maven projects from AI answers, one in the folder bikeshare-poor and one in the folder bikeshare-good.

1. Start a new chat in your AI tool. Copy Prompt B from prompts.md, replace the line <paste the BikeShare specification here> with the seven lines of The BikeShare specification, which sit between the lines ```text and ``` of that section, and send it.
2. In VS Code, right-click an empty area of the Explorer, click New Folder, and name it bikeshare-poor. For each file in the answer, right-click bikeshare-poor, click New File, and type the path that the answer gives, such as pom.xml or src/main/java/bikeshare/Main.java. VS Code creates the folders in the path. Paste the code and save the file.
3. In the terminal, run the commands below from the repository folder. In the java command, use the package and the class whose file contains public static void main, such as bikeshare.Main. Try a few rentals and returns, then type quit.

```
cd bikeshare-poor
mvn package
java -cp target/classes bikeshare.Main
cd ..
```

4. If mvn package fails, paste the error into the same chat, ask the AI to fix it, update the files, and note this in the report.
5. Repeat steps 1 to 4 in another new chat with Prompt C and the folder bikeshare-good.
6. Click View, then Command Palette, run Java: Import Java Projects into Workspace, and wait for Java: Ready, so that VS Code loads the two new projects.
7. Double-click every Java file of bikeshare-poor, click View, then Problems, and type bikeshare-poor in the filter box of the Problems panel, so that it shows only the issues of that project. Count the SonarQube issues in total and for each rule, and leave out the warnings from the Java extension, as in step C3. Do the same for bikeshare-good. If a file shows the non-project warning of step C3, open that project folder by itself in the same way, and open your repository folder again afterwards.
8. Run Prompt A in a new chat for each version, with <folder> replaced by the folder name and the version's Java files attached. Save the answers as reviews/bikeshare-poor-review.md and reviews/bikeshare-good-review.md.
9. Commit and sync, fill the table of Task 4 in the report, and answer questions 4.1 to 4.3.

Part G. Task 5, Reflection

Answer questions 5.1 to 5.3 in the report, in two to four sentences each.

Part H. Submit

1. Fill the line Repository URL and the Tools table at the top of the report. Take the JDK and Maven versions from the commands of Part A. For VS Code, click Help, then About on Windows, or Code, then About Visual Studio Code on macOS. For SonarQube for IDE, click View, then Extensions, click SonarQube for IDE, and copy the version number shown next to its name.
2. Fill the AI usage declaration at the end of the report.
3. Commit and sync. Open your repository on GitHub, and check that it shows parcelpoint, bikeshare-poor, bikeshare-good, reviews and LAB_REPORT.md, and that it shows no target folder. Then open its URL in a private or incognito window of your browser, where you are not signed in, to check that anyone can see it.
4. On MyCourses, open the lab assignment, paste the URL of your repository, upload LAB_REPORT.md, and submit before Sunday 25 October 2026 at 23:55.

If Something Goes Wrong

- If the terminal says that java, mvn or git is not recognized, close the terminal and open a new one, or restart VS Code, so that it reads the new settings. If the message stays, repeat the matching step of Part A.
- If mvn -version or mvn package reports a Java version below 17, Maven is still using an older JDK. Run the installer of step A1 again, choose Set JAVA_HOME variable on Windows, and open a new terminal.
- If the status bar shows Java: Error or Java: No Projects Imported, click View, then Command Palette, run Java: Clean Java Language Server Workspace, and confirm the restart.
- If the Problems panel shows no SonarQube issue for an open Java file, click View, then Command Palette, and run SonarQube: Analyze Current File with SonarQube.
- If Publish Branch or Sync Changes asks you to sign in, sign in to GitHub in the browser, and allow VS Code to use your account.